

Remember Smarter: Visual History Compressor and Hyperbolic Experience Space for Robotic Memory

Dai Zhou¹, Jiexi Yan^{1,†}, Tong Li¹, Yuxuan Wang¹, Cheng Deng^{1,†}

¹Xidian University

[†]Corresponding author: yanjiexi@xidian.edu.cn, chdeng@mail.xidian.edu.cn

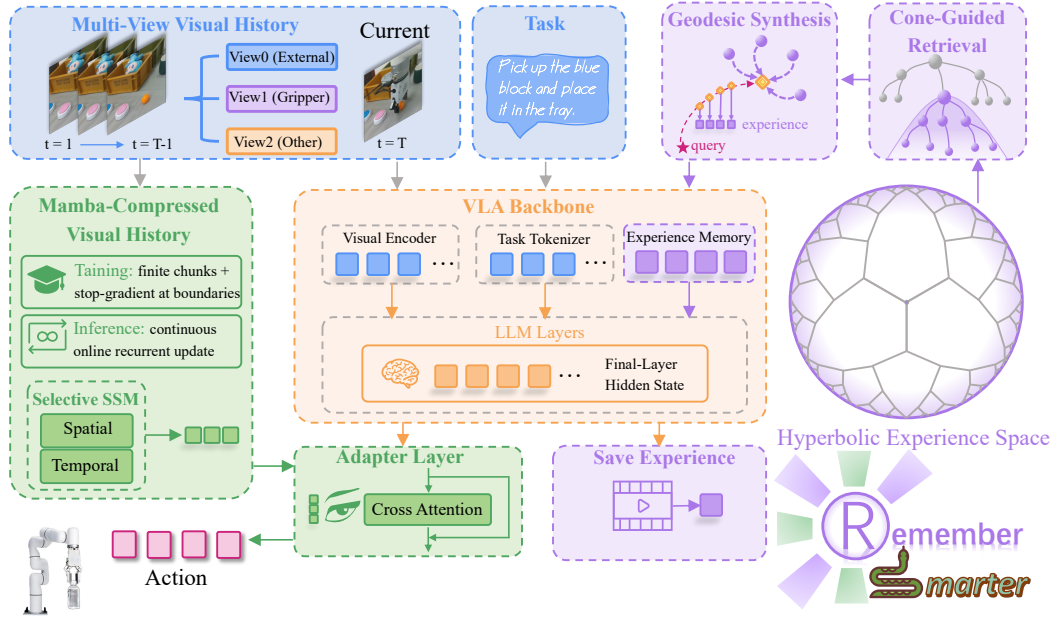


Figure 1: High-level concept of Remember Smarter. The fine-grained branch compresses multi-view visual patch history, while the coarse-grained branch retrieves structured hyperbolic experience.

Abstract

Long-horizon robot policies require compact access to recent observations and reusable experience without expanding the vision-language-action (VLA) context. We introduce Remember Smarter (RS), a plug-and-play module with complementary visual-history and hyperbolic experience-memory branches. Its visual branch compresses multi-view patch histories using bidirectional spatial Mamba and causal temporal Mamba, then exposes the resulting memory to action-facing hidden states through residual cross-attention while leaving the VLM visual-token stream unchanged. Its experience branch stores successful final-layer VLM states in a Poincare VAE space, organizes them hierarchically, and asynchronously converts retrieved experience into geodesic prompt tokens without blocking action inference. When adapted to π_0 , RS increases total success on LIBERO-Plus from 53.6% to 70.6% and achieves substantial performance gains in real-robot experiments designed to evaluate memory retention and experience utilization.

1 Introduction

Vision-language-action (VLA) models ground language instructions in visual observations to predict robot actions [1, 2, 3, 4]. Their memory, however, remains fragile on long-horizon tasks. Supplying more historical frames to the backbone is costly and can be noisy: redundant observations consume the token budget and can obscure the sparse state changes needed for decision-making [5, 6]. In addition, a flat memory bank neither represents hierarchical abstractions explicitly nor avoids global retrieval from a growing set of demonstrations [7, 8, 9].

We propose **Remember Smarter (RS)**, a plug-and-play memory module that combines visual-history compression with hyperbolic experience retrieval. For fine-grained memory, RS compresses multi-view visual patch histories using spatial and temporal Mamba blocks [10, 11]. Consecutive training fragments pass recurrent state across their boundaries while stopping gradients every 16 frames; at inference, the recurrent state is updated online. A residual cross-attention adapter lets action-facing hidden states read the ordered history memory before the action projection without modifying VLM visual tokens. For coarse-grained memory, RS collects only successful training episodes and stores the final action-facing VLM representation from each episode. A Poincare variational autoencoder (P-VAE) supplies retrieval coordinates. An equal-radius auxiliary copy of the leaves is optimized with a compact triplet objective inspired by Hyperbolic Hierarchical Clustering (HypHC) [12] and decoded into a binary topology. A non-blocking beam search retrieves up to eight P-VAE leaves, finds their lowest common internal ancestor, and uses that ancestor’s P-VAE-space prototype to guide four geodesic prompt tokens appended to the VLM input.

Our contributions are summarized as follows:

- We propose **Remember Smarter (RS)**, a plug-and-play memory architecture that augments VLA policies with complementary online visual-history memory and hierarchical experience memory without modifying the VLA backbone.
- We introduce an online visual-history compressor based on spatial and temporal Mamba, with state-passing truncated backpropagation through time (TBPTT) during training and fixed-state causal streaming at inference.
- We introduce a hierarchical hyperbolic experience space that combines P-VAE retrieval coordinates, binary-tree construction, entailment-cone routing, and geodesic experience-token generation.

2 Related Works

2.1 VLA Memory Mechanisms

VLA policies combine visual perception, language grounding, and action generation in a single model [2, 4, 1]. Recent lightweight designs improve the perception–action interface or reduce backbone cost [13, 14, 15, 16], but long-horizon control also requires state that persists beyond one observation. Existing memory-oriented policies use visual memory [6, 17], phase or object-state modeling [18, 19], demonstration-derived prompts [20], or multi-scale perceptual, cognitive, and language memory [21, 22, 23]. RS instead couples a fixed-state visual compressor with an asynchronously queried experience space, assigning distinct computational roles to fine- and coarse-grained memory.

Hierarchical organization has also been used to structure lifelong robot experience [24, 25] and multi-memory planning [26]. Related work studies hierarchical manipulation concepts [27] or action trees for manipulation world models [28]. In contrast, RS uses a decoded hierarchy only as a retrieval index over successful policy representations, not as an explicit symbolic task tree for action planning.

2.2 State Space Models and Mamba

Selective SSMs, including Mamba and its structured-state-space dual form, offer linear-time recurrent sequence processing [10, 11]. Their long-context behavior nevertheless depends on state capacity, initialization, and inter-layer information flow [29, 30, 31]; SSM reduction has also been studied through balanced truncation [32]. For long video, STORM and state-space hierarchical compression use temporal SSM encoders to preserve relevant dynamics before reducing visual tokens [33, 34].

RoboMamba applies Mamba directly within a VLA policy [35]. RS uses Mamba differently: it retains the pre-trained VLA backbone and compresses only the multi-view patch history into a small state that the action-facing representations can read.

2.3 Hyperbolic Geometry for Hierarchical Representation

Poincare embeddings and Lorentz-model learning established hyperbolic geometry as a useful inductive bias for latent hierarchies [36, 37]. Poincare VAEs, entailment cones, and differentiable hyperbolic clustering respectively provide latent-variable, directed-relation, and tree-construction tools [8, 9, 12]. RS uses P-VAE coordinates for retrieval while optimizing a separate auxiliary copy only to decode the hierarchy. This separation preserves the policy-facing retrieval space and avoids treating the hierarchy-learning coordinates as online query features.

3 Preliminary

3.1 Problem statement

We consider long-horizon manipulation with a pre-trained VLA backbone. At time t , the robot receives a language instruction L , a multi-view image set $I_t \in \mathbb{R}^{V \times H \times W \times 3}$, and the preceding visual history $\mathcal{H}_{1:t-1} = \{I_1, \dots, I_{t-1}\}$. The visual-history compressor uses this history as an auxiliary memory branch. The policy predicts $a_t = \pi_\theta(I_t, L, M_t, \mathcal{E})$, where M_t is compact visual memory and $\mathcal{E}$ is reusable experience from prior demonstrations. For manipulation, a_t is typically a 6-DoF end-effector pose, optionally augmented with a gripper state for 7-DoF control.

3.2 State Space Models

State Space Models (SSMs) map a one-dimensional input sequence $x(t) \in \mathbb{R}^L$ to an output sequence $y(t) \in \mathbb{R}^L$ through a hidden state $h(t) \in \mathbb{R}^N$. A continuous SSM is defined by the state matrix $\mathbf{A}$, input matrix $\mathbf{B}$, and output matrix $\mathbf{C}$ [10, 11]:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t). \quad (1)$$

Modern SSMs such as Mamba operate on discrete sequences. With timescale parameter Δ , zero-order-hold discretization converts $\mathbf{A}$ and $\mathbf{B}$ into

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \quad (2)$$

$$\bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - I) \cdot \Delta\mathbf{B}, \quad (3)$$

which yields the recurrent update

$$h_t = \bar{\mathbf{A}}h_{t-1} + \bar{\mathbf{B}}x_t, \quad y_t = \mathbf{C}h_t. \quad (4)$$

Mamba instantiates this recurrent form as a selective SSM block. Rather than using fixed transition and projection parameters for every token, it predicts input-dependent parameters, including step size Δ_t and token-wise projections $\mathbf{B}_t$ and $\mathbf{C}_t$. The resulting selective scan processes a sequence in linear time, summarizes preceding tokens in a compact state, and supports a parallel scan during training or recurrent state updates at inference. For visual history, redundant frames can be absorbed into the state while task-relevant changes still update the memory representation.

This recurrent view motivates state-passing truncated backpropagation through time (TBPTT). During training, Mamba processes consecutive fragments of multi-view visual patch tokens. The stop-gradient operator preserves the forward state while limiting gradients to the current fragment. Our default setting uses TBPTT of 16 and histories of at most 64 frames. We separately ablate history lengths of 8, 16, 32, 64, and 128 frames and TBPTT truncation lengths of 4, 8, 16, 24, and 32 frames. At inference, the same constant-shape recurrent state is updated online without storing the full visual history. Appendix C.2 gives the optimization details.

3.3 Hyperbolic Space and Entailment Cones

We briefly introduce the geometric concepts underlying hierarchical experience representation, including hyperbolic space, the Poincare ball model, geodesics, and entailment cones. This section focuses on their intuitive meanings and relationships; the corresponding metrics, geometric operators, and formal definitions are provided in Appendix B.2.

Hyperbolic geometry. Hyperbolic space is a non-Euclidean space with constant negative curvature. Its available volume grows exponentially with distance from the origin, resembling the exponential increase in the number of nodes across successive levels of a tree. This property makes hyperbolic space particularly suitable for representing hierarchical data: general concepts can occupy a compact central region, while increasingly specific concepts can spread over the much larger outer regions. Compared with Euclidean embeddings, hyperbolic embeddings can therefore preserve multi-level hierarchical relationships with relatively low distortion [36, 37].

Poincare ball model. The Poincare ball is a conformal model of hyperbolic space in which all points lie inside an open unit ball. Although the representation is bounded in Euclidean coordinates, the boundary is infinitely far away under the hyperbolic metric. Distances are increasingly stretched toward the boundary, allowing the ball to accommodate a large number of well-separated representations. Its radial organization also provides an intuitive interpretation of abstraction: points near the origin generally represent broad or abstract concepts, whereas points closer to the boundary represent more specific concepts. Because the model is conformal, local angles are preserved, which facilitates reasoning about both distance and direction.

Geodesics. A geodesic is the shortest path between two points under the hyperbolic metric. It is the hyperbolic counterpart of a straight line in Euclidean space. In the Poincare ball, geodesics are represented by diameters of the ball or circular arcs that intersect its boundary orthogonally. Moving along a geodesic provides a geometry-consistent interpolation between two representations. Points sampled at different positions on the path capture progressively changing levels of proximity to its endpoints without leaving the underlying hyperbolic manifold.

Entailment cones. Entailment cones extend hyperbolic embeddings with directed hierarchical relations. Each cone originates from an apex representing a relatively general concept and covers a region associated with its more specific descendants. A candidate representation is entailed by the apex when it lies inside this directed region. Cone membership therefore encodes a directed ancestor–descendant containment relation between the apex and the candidate, rather than merely their undirected similarity. The cone aperture varies with the location of its apex, enabling the geometry to express different levels of generality and specificity while preserving transitive hierarchical relationships [9].

4 Remember Smarter

4.1 Overview

Figure 1 summarizes RS, which augments a pre-trained VLA policy with a visual-history compressor and a hierarchical experience memory. The visual pathway summarizes changes in multi-view observations; the experience pathway retrieves reusable patterns from successful demonstrations. Together, they provide fine- and coarse-grained memory without repeatedly extending the VLA context with raw history.

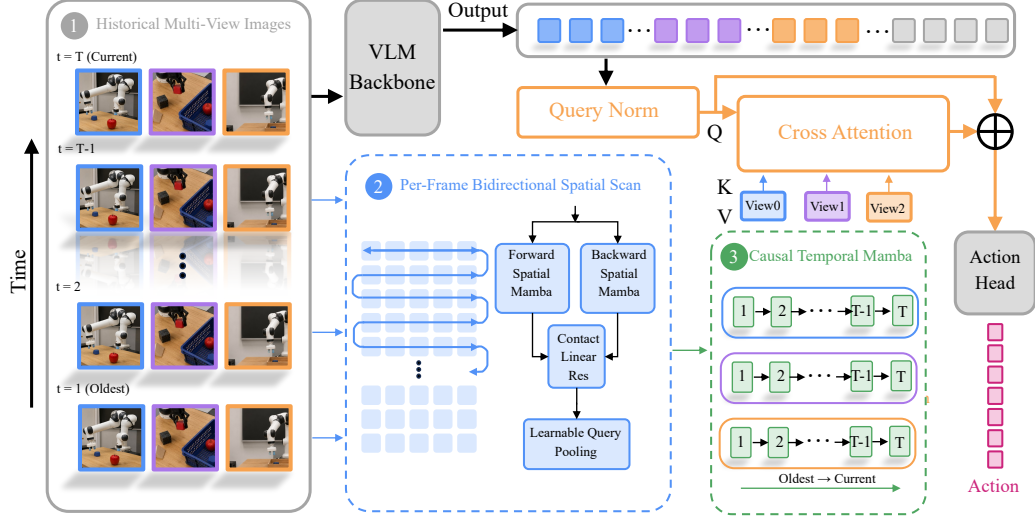


Figure 2: Mamba-compressed visual-history pipeline. Historical multi-view patch features are extracted by the shared visual encoder and mapped into a unified feature space by a lightweight adapter. Each frame is encoded through bidirectional spatial scans and learned-query pooling, after which causal temporal Mamba layers recurrently aggregate the per-view history. The resulting visual-history representations provide the keys and values for residual cross-attention, while the final hidden representations used for action prediction provide the queries. The enhanced hidden representations are then passed to the action head.

4.2 Mamba-Compressed Visual History

As illustrated in Figure 2, the visual-history branch converts a multi-view image stream into a compact recurrent memory for action prediction. It reuses the VLA visual encoder, adapts its patch features to a shared latent width, applies bidirectional spatial Mamba and learned-query pooling within each frame, and then aggregates each view chronologically with causal temporal Mamba. A residual cross-attention adapter lets the action-facing hidden states read this memory without modifying the VLA visual-token stream. Appendix B.1 specifies the backbone paths, dimensions, masks, and training configuration.

Visual feature extraction and adaptation. Let E_{vis} denote the visual encoder already present in the VLA policy. The current and historical images share this encoder, avoiding a second visual backbone in the memory pathway. For view v at time τ , its patch grid is adapted to the Mamba width d by a lightweight module A_ϕ :

$$F_\tau^v = E_{\text{vis}}(I_\tau^v), \quad X_\tau^v = A_\phi(F_\tau^v) \in \mathbb{R}^{H_p \times W_p \times d}. \quad (5)$$

The adapter decouples the compressor from the native width and representation format of a particular visual encoder, allowing the memory architecture to be attached to VLA backbones with different visual feature dimensions.

Per-frame bidirectional serpentine encoding. For each frame and view, the adapted patch grid is linearized by a serpentine permutation π : odd-numbered rows are traversed from left to right, whereas even-numbered rows are traversed from right to left. This alternating direction keeps consecutive patches spatially adjacent when the scan crosses a row boundary. With $N_p = H_p W_p$, the resulting sequence is

$$x_\tau^v = (X_\tau^v[\pi(1)], \dots, X_\tau^v[\pi(N_p)]) \in \mathbb{R}^{N_p \times d}. \quad (6)$$

A forward spatial Mamba follows this sequence, whereas a backward spatial Mamba follows its exact reverse. We reverse the latter output back to the forward index system before fusion:

$$\begin{aligned}\vec{Y}_\tau^v &= \text{Mamba}_{\text{sp}}^\rightarrow(x_\tau^v), \\ \overleftarrow{Y}_\tau^v &= \text{Rev}(\text{Mamba}_{\text{sp}}^\leftarrow(\text{Rev}(x_\tau^v))), \\ U_\tau^v &= \frac{1}{2}(\vec{Y}_\tau^v + \overleftarrow{Y}_\tau^v).\end{aligned}\tag{7}$$

The two directions preserve aligned patch positions while providing complementary spatial context. A learned query $q \in \mathbb{R}^d$ then pools the fused sequence, and a learned projection A_{pool} produces a frame-level representation:

$$\alpha_\tau^v = \text{softmax}_{N_p}\left(\frac{U_\tau^v q}{\sqrt{d}}\right), \quad p_\tau^v = A_{\text{pool}}((\alpha_\tau^v)^\top U_\tau^v).\tag{8}$$

Causal temporal aggregation. For each view, the pooled frame representations are processed chronologically by two causal temporal Mamba layers:

$$(m_\tau^v, s_\tau^v) = \text{Mamba}_{\text{temp}}^{(2)}(p_\tau^v, s_{\tau-1}^v), \quad \tau = 1, \dots, t.\tag{9}$$

Here, the superscript in $\text{Mamba}_{\text{temp}}^{(2)}$ denotes the stacked two-layer temporal module. Thus, m_τ^v depends only on observations available up to time τ . Temporal parameters are shared across views, while each view maintains its own recurrent state s_τ^v . The current multi-view memory has one ordered representation for each view,

$$M_t = \text{stack}_{v \in \mathcal{V}}(m_t^v) \in \mathbb{R}^{B \times V \times d}.\tag{10}$$

This ordered history tensor serves as the memory source for the residual cross-attention described below. The state-update rule for unavailable views and the ordering convention are specified in Appendix B.1.

Residual history cross-attention. Let $H_t \in \mathbb{R}^{B \times Q \times D_h}$ denote the final hidden representations used for action prediction. RS applies one residual cross-attention adapter,

$$\begin{aligned}Q_t &= H_t W_Q^{\text{att}}, \\ K_t &= M_t W_K^{\text{att}}, \\ V_t &= M_t W_V^{\text{att}}, \\ \Delta H_t &= \text{Attn}(Q_t, K_t, V_t) W_O^{\text{att}}, \\ \tilde{H}_t &= H_t + \gamma \Delta H_t,\end{aligned}\tag{11}$$

where Attn is multi-head scaled dot-product attention. The residual scale is $\gamma = 0.05$. The Mamba memory supplies keys and values, action-facing hidden states supply queries, and the original action head consumes the enhanced representation. Appendix B.1 details masking and backbone-specific insertion points.

Training and streaming inference. During training, each trajectory is divided into consecutive fragments. The temporal state is passed forward within an episode, while the stop-gradient operator detaches it at each TBPTT boundary:

$$(Y_k^v, S_k^v) = \text{Mamba}_{\text{temp}}^{(2)}(P_k^v, \text{stopgrad}(S_{k-1}^v)).\tag{12}$$

In Eq. 12, P_k^v is the sequence of pooled frame representations in fragment k , S_{k-1}^v is the incoming state, and Y_k^v and S_k^v are the output sequence and updated state. At inference, the same fixed-shape recurrent state is updated one frame at a time, so prior images and patch tokens need not be retained.

4.3 Hyperbolic Experience Space

RS stores reusable experience in a hyperbolic hierarchy and retrieves it asynchronously. Offline memory consists of P-VAE coordinates for successful demonstrations and a HypHC-inspired binary topology over those coordinates. At inference, a background worker queries this hierarchy and converts retrieved experience into a fixed number of prompt tokens. Appendix B.2 gives the formal geometry and implementation details.

Experience storage. After policy training, RS collects final action-facing hidden representations h_i from successful demonstrations. A Poincare variational autoencoder (P-VAE) [8] maps each h_i to posterior mean z_i in the Poincare ball. These means are retrieval leaves, and the P-VAE decoder maps selected latent points back to the policy-representation space. Failed demonstrations are not stored in experience memory.

Hierarchical organization. RS builds a separate binary hierarchy over the stored leaves with a compact objective inspired by HypHC [12]. Unlike a flat memory bank, the hierarchy exposes nested experience groups for coarse-to-fine routing. This stage optimizes an auxiliary copy of the leaf coordinates and therefore leaves the P-VAE retrieval coordinates unchanged. After decoding the tree, each internal prototype is initialized by the Karcher mean[38, 39] of its descendant leaves, which minimizes their aggregate squared hyperbolic distance. Entailment-cone calibration then provides directed parent-to-descendant regions, complementing the symmetric hyperbolic distance used for candidate ranking.

Experience-space visualization. Figure 3 visualizes the P-VAE coordinates and corresponding HypHC hierarchy for the first 300 successful π_0 inference records from the released physical-intelligence/libero dataset. We encode each record’s final-layer VLM hidden state and plot the resulting leaves from its ten tasks (T00–T09, listed in the appendix). The left panel shows P-VAE coordinates and the right panel the decoded binary hierarchy. Colors identify the source task, and the black star denotes the virtual root. This qualitative plot is not used for online retrieval, which uses the full P-VAE coordinates and decoded tree.

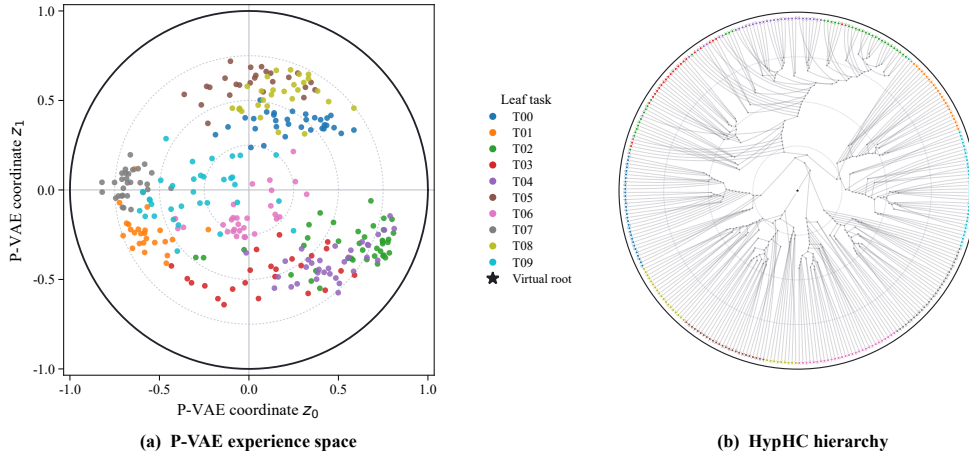


Figure 3: P-VAE experience space and HypHC hierarchy for the first 300 successful π_0 inference records from <https://huggingface.co/datasets/physical-intelligence/libero>. Each colored point is a final-layer VLM hidden state encoded for visualization; color denotes its source task T00–T09. The black star in (b) marks the virtual root of the decoded HypHC hierarchy.

Asynchronous hierarchical retrieval. At time step t , the current hidden representation is encoded as a query q_t in the same P-VAE space. A non-blocking worker traverses the binary tree, using hyperbolic distance and the entailment cones of internal prototypes to prioritize relevant subtrees before ranking candidate leaves. It returns at most eight leaves, which bounds the subsequent composition cost. The action loop never waits for this worker; queue management and reset handling are specified in Appendix B.2.

Geodesic experience-token generation. The retrieved leaves are summarized by their lowest common internal ancestor a_t in the binary hierarchy. Let p_{a_t} denote the corresponding prototype in the P-VAE space. RS samples four points along the geodesic from the query to this shared prototype,

$$u_j = \text{Exp}_{q_t}(\tau_j \text{Log}_{q_t}(p_{a_t})), \quad e_j = D_\psi(u_j), \quad j = 1, \dots, 4. \quad (13)$$

Table 1: Robustness on the full LIBERO-Plus benchmark. Entries are strict success rates (%), and bold indicates the highest score in each reported column. Total is computed over all 10,030 tasks rather than averaging the seven dimension-level scores. Published baselines retain the settings reported in their original work; the MemoryVLA variants are evaluated in the zero-shot setting [22]. The final row is the absolute change of π_0 +RS relative to π_0 .

Method	Camera	Robot	Language	Light	Background	Noise	Layout	Total
MemoryVLA++	36.4	68.9	88.7	93.8	90.6	63.5	83.8	73.1
MemoryVLA	42.7	44.9	84.4	92.8	95.0	62.1	84.7	70.2
OpenVLA-OFT	56.4	31.9	79.5	88.7	93.3	75.8	74.2	69.6
π_0 -Fast	65.1	21.6	61.0	73.2	73.2	74.4	68.8	61.6
π_0	13.8	6.0	58.8	85.0	81.4	79.0	68.9	53.6
π_0 +RS	50.8	45.8	65.3	92.1	94.0	81.2	78.0	70.6
Δ over π_0	$\uparrow 37.0$	$\uparrow 39.8$	$\uparrow 6.5$	$\uparrow 7.1$	$\uparrow 12.6$	$\uparrow 2.2$	$\uparrow 9.1$	$\uparrow 16.9$

where $0 < \tau_1 < \dots < \tau_4 < 1$. Decoding these points produces four experience tokens that progress from the current query toward the retrieved shared abstraction. Sampling this geodesic supplies a fixed prompt budget that is independent of the number of stored experiences. The four virtual $\langle \text{exp} \rangle$ slots are inserted immediately before the first $\langle \text{img} \rangle$ embedding. A learned [MEM_QUERY] slot follows the task prompt, and its final hidden representation forms the query for the next retrieval. The original VLM self-attention fuses the added tokens with the visual and language context; no separate experience-fusion module is used.

5 Experiments

We evaluate RS only with π_0 in four settings: simulation on LIBERO-Plus, real-robot experiments, component analysis of visual compression and hyperbolic experience retrieval on standard LIBERO, and an analysis of how the Mamba training-history window affects optimization and online compression.

5.1 Simulation Benchmarks

Following LIBERO-Plus [40], we evaluate robustness over seven perturbation dimensions: camera viewpoint, robot initial state, language instruction, lighting, background, sensor noise, and object layout. Both π_0 and π_0 +RS start from the official π_0 -LIBERO checkpoint: the foundation model is pre-trained on large-scale robot data and fine-tuned on the original LIBERO demonstrations. RS is adapted only with those original LIBERO demonstrations; it is not trained on LIBERO-Plus trajectories. We evaluate each of the 10,030 LIBERO-Plus task instances once. Table 1 reports strict success rates over the full benchmark. RS raises total success from 53.6% to 70.6%, a 16.7-point increase. Its total is 2.5 points below the 73.1% reported for MemoryVLA++ in the zero-shot setting [22].

5.2 Real-Robot Memory Tasks

We evaluate π_0 and π_0 +RS across seven real-robot task conditions; their detailed definitions are given in Appendix C.5. The suite tests four memory-centric dimensions: repeated-action counting (Press-2/4/6), ordered event sequences (Order), object-name alignment (Chenchen), and compositional long-horizon execution (Chen.-H). Occluded placement provides an additional perceptual stressor. We highlight the compositional *Chenchen-hard* (Chen.-H) task in the main text: *First, move Chenchen from the yellow box to the green box. Then press the blue button once. Next, put the yellow ball into the green cup, and place the green cup into the yellow box. Finally, press the pink button once.* The task requires retaining object identity and recognizing the completion status of multiple subgoals. RS is trained only on the simpler conditions and receives no additional Chen.-H training.

Table 2 reports higher strict success for RS in six of the seven conditions. On held-out Chen.-H, success rises from 20% for π_0 to 80% for π_0 +RS. The only decrease is on short-horizon Chen.-S (64% to 60%); the largest increase is on Chen.-H (+60 points).

Table 2: Real-robot strict success rate (%). Press- k denotes k repeated presses; Order denotes ordered button pressing; Occl. denotes occluded placement; Chen.-S/H denote the simple/hard Chenchen tasks.

Method	Press-2	Press-4	Press-6	Order	Occl.	Chen.-S	Chen.-H
π_0	64	20	0	20	72	64	20
π_0 +RS	92	70	60	40	88	60	80

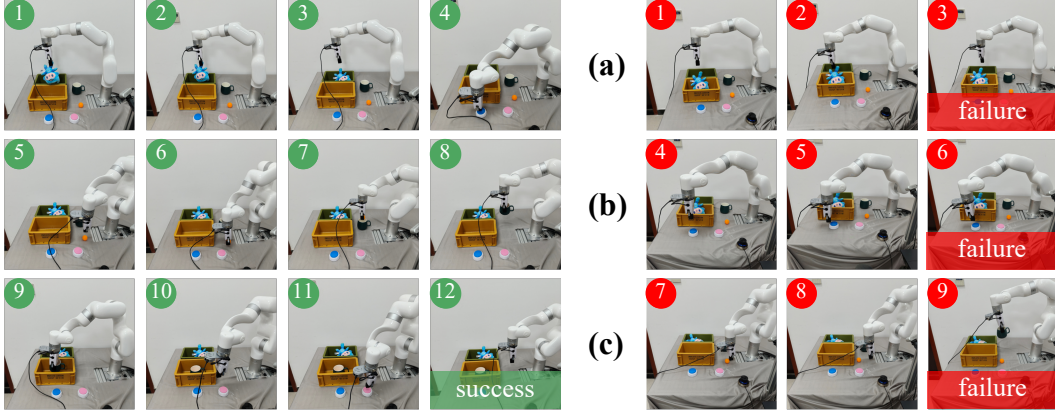


Figure 4: Representative real-robot rollouts on the compositional Chen.-H task. The green sequence shows a successful π_0 +RS rollout. The red sequences show π_0 failures: (a) failure to lift Chenchen, (b) failure to press the required button, and (c) failure to place the ball in the cup. These examples are consistent with the difficulty of recognizing a failed subtask and recovering without access to successful experience.

5.3 Ablations and Analysis

Component contributions. We compare the base π_0 , π_0 +VMC, and full π_0 +RS on the four standard LIBERO suites [41]. VMC contains only the visual Mamba compressor; RS additionally enables the Poincare experience space. As shown in Figure 5, VMC raises Long success from 85.2% to 92.6%, showing the value of visual-history compression. Adding experience memory further raises Long to 94.2% and the four-suite overall score to 96.2%, a 2.1-point improvement over π_0 . The gain is concentrated on the long-horizon suite: RS preserves comparable performance on the short Spatial, Object, and Goal suites while yielding the largest improvement on Long.

History length and truncated backpropagation. Figure 6(a) measures LIBERO-Long success-rate change relative to no Mamba every 5k training steps. Histories of 64 and 128 frames both reach roughly a 6%–7% improvement by 20k–25k steps, whereas shorter histories remain weaker. Under this study, a 64-frame Mamba history provides similar extrapolation to 128 frames without the longer context. Figure 6(b) fixes the history at 64 frames and varies the TBPTT gradient-truncation length. TBPTT=16 has the fastest sustained loss reduction and the lowest loss at 25k steps; shorter windows plateau earlier, while longer windows do not improve convergence.

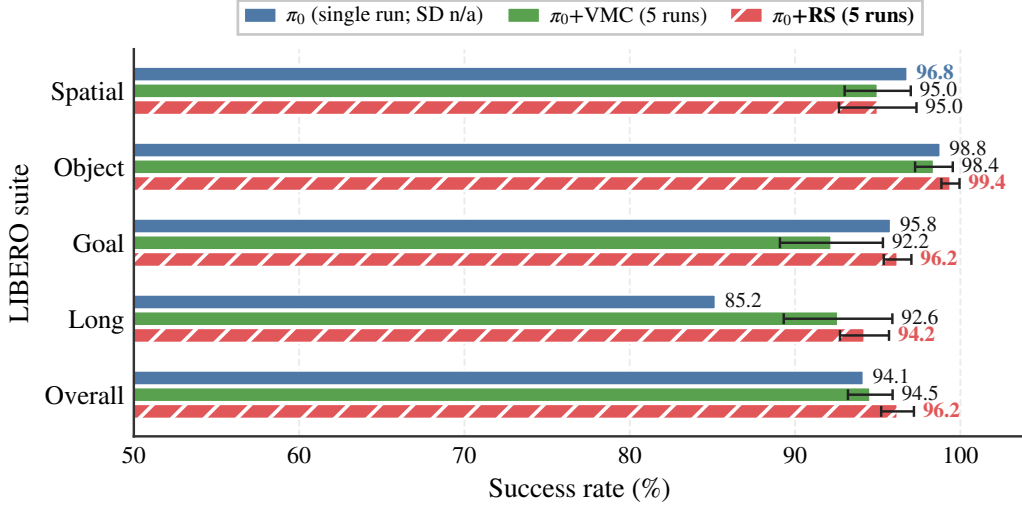


Figure 5: Component ablation on the four standard LIBERO suites. VMC is the visual Mamba compressor without the Poincare experience space. π_0 is a single run (standard deviation unavailable); π_0 +VMC and π_0 +RS use five runs, and error bars denote standard deviation.

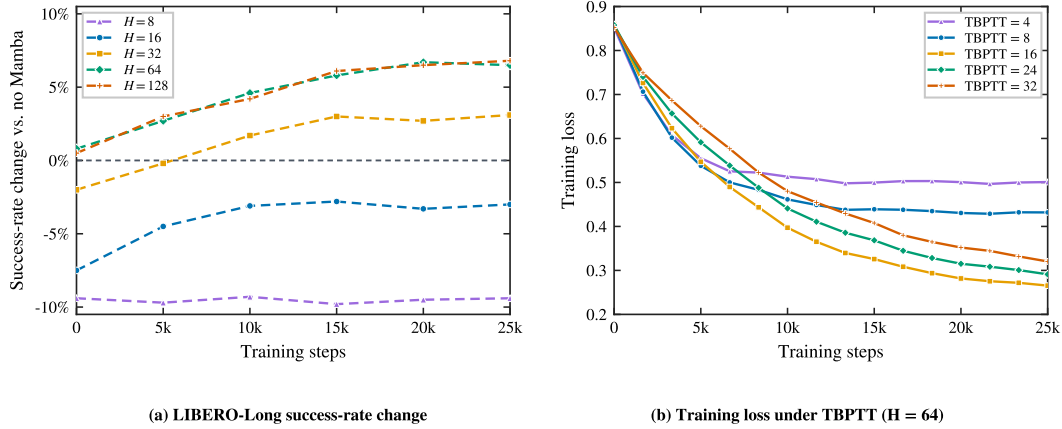


Figure 6: Effect of Mamba history length and TBPTT truncation. (a) Long success-rate change relative to a no-Mamba baseline, measured every 5k training steps. (b) Training loss for a fixed 64-frame history under different TBPTT truncation lengths.

6 Conclusion

We presented Remember Smarter, a plug-and-play memory framework that augments VLA policies with fine-grained visual-history compression and hierarchical experience retrieval. The visual branch reuses the VLA visual encoder, applies bidirectional spatial and causal temporal Mamba, and maintains a compact multi-view history in fixed-shape recurrent state. This memory supplies keys and values to a residual cross-attention reader, while action-facing hidden states supply queries and the original action head consumes the result.

The experience branch maps successful policy representations to P-VAE leaves and organizes them with a separate HypHC-inspired binary topology. Karcher-mean initialization and entailment-cone calibration yield internal prototypes for routing. At inference, a non-blocking beam retrieves up to eight leaves; their lowest common internal prototype guides four geodesic experience tokens. The

worker remains outside the action-critical path. On the selected LIBERO-Plus subset, RS raises total success from 53.6% to 70.3%. It also achieves substantial performance gains in real-robot experiments designed to evaluate memory retention and experience utilization. These results support the value of combining compact online visual history with asynchronously retrieved successful experience for the evaluated long-horizon settings.

References

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2026.
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control, 2023.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-1: Robotics transformer for real-world control at scale, 2023.
- [4] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model, 2024.
- [5] Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies, 2026.
- [6] Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, and Jiafei Duan. SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation, 2025.
- [7] Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. MemER: Scaling up memory for robot control via experience retrieval, 2025.
- [8] Emile Mathieu, Charline Le Lan, Chris J. Maddison, Ryota Tomioka, and Yee Whye Teh. Continuous hierarchical representations with poincare variational auto-encoders. In *Advances in Neural Information Processing Systems*, 2019.
- [9] Octavian-Eugen Ganeva, Gary Becigneul, and Thomas Hofmann. Hyperbolic entailment cones for learning hierarchical embeddings. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.
- [11] Tri Dao and Albert Gu. Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality, 2024.
- [12] Ines Chami, Albert Gu, Vaggos Chatziafratis, and Christopher Re. From trees to continuous embeddings and back: Hyperbolic hierarchical clustering. In *Advances in Neural Information Processing Systems*, 2020.
- [13] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025.
- [14] Tao Lin, Yilei Zhong, Yuxin Du, Jingjing Zhang, Jiting Liu, Yinxinyu Chen, Encheng Gu, Ziyang Liu, Hongyi Cai, Yanwen Zou, et al. Evo-1: Lightweight vision-language-action model with preserved semantic alignment, 2025.
- [15] Yuankai Luo, Woping Chen, Tong Liang, Baiqiao Wang, and Zhenguo Li. SimVLA: A simple VLA baseline for robotic manipulation, 2026.
- [16] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, et al. VLA-Adapter: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

- [17] Yijing Zhou, Qiwei Liang, Sitong Zhuang, Jiaxi Li, Xianpeng Wang, Boyang Cai, Yunyang Mo, and Renjing Xu. Action-effect memory pretraining for robot manipulation, 2026.
- [18] Yiguo Fan, Pengxiang Ding, Shuanghao Bai, Xinyang Tong, Yuyang Zhu, Hongchao Lu, Fengqi Dai, Wei Zhao, Yang Liu, Siteng Huang, et al. Long-VLA: Unleashing long-horizon capability of vision language action model for robot manipulation, 2025.
- [19] Nhat Chung, Taisei Hanyu, Toan Nguyen, Huy Le, Frederick Bumgarner, Duy Minh Ho Nguyen, Khoa Vo, Kashu Yamazaki, Chase Rainwater, Tung Kieu, et al. Rethinking progression of memory state in robotic manipulation: An object-centric perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 3407–3415, 2026.
- [20] Runhao Li, Wenkai Guo, Zhenyu Wu, Changyuan Wang, Haoyuan Deng, Zhenyu Weng, Yap-Peng Tan, and Ziwei Wang. MAP-VLA: Memory-augmented prompting for vision-language-action model in robotic manipulation, 2025.
- [21] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation, 2025.
- [22] Hao Shi, Weiye Li, Bin Xie, Yulin Wang, Renping Zhou, Tiancai Wang, Xiangyu Zhang, Ping Luo, and Gao Huang. MemoryVLA++: Temporal modeling via memory and imagination in vision-language-action models, 2026.
- [23] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, et al. MEM: Multi-scale embodied memory for vision language action models, 2026.
- [24] Leonard B"armann, Chad DeChant, Joana Plewnia, Fabian Peller-Konrad, Daniel Bauer, Tamim Asfour, and Alex Waibel. Episodic memory verbalization using hierarchical representations of life-long robot experience. In *IEEE-RAS International Conference on Humanoid Robots*, pages 783–790, 2025.
- [25] Jauwairia Nasir, Deok-Hwa Kim, and Jong-Hwan Kim. ART neural network-based integration of episodic memory and semantic memory for task planning for robots. *Autonomous Robots*, 43(8):2163–2182, 2019.
- [26] Yanfang Zhou, Xiaodong Li, Yuntao Liu, Yongqiang Zhao, Xintong Wang, Zhenyu Li, Jinlong Tian, and Xinhai Xu. M2PA: A multi-memory planning agent for open worlds inspired by cognitive theory. In *Findings of the Association for Computational Linguistics: ACL*, 2025.
- [27] Ruizhe Liu, Pei Zhou, Qian Luo, Li Sun, Jun Cen, Yibing Song, and Yanchao Yang. HiMaCon: Discovering hierarchical manipulation concepts from unlabeled multi-modal data. In *Advances in Neural Information Processing Systems*, 2025.
- [28] Ying Li, Xiaobao Wei, Xiaowei Chi, Yuming Li, Zhongyu Zhao, Hao Wang, Ningning Ma, Ming Lu, and Shanghang Zhang. ManipDreamer: Boosting robotic manipulation world model with action tree and visual guidance, 2025.
- [29] Yingfa Chen, Xinrong Zhang, Shengding Hu, Xu Han, Zhiyuan Liu, and Maosong Sun. Stuffed mamba: State collapse and state capacity of RNN-based long-context modeling, 2024.
- [30] Shida Wang. LongSSM: On the length extension of state-space models in language modelling, 2024.
- [31] Wei He, Kai Han, Yehui Tang, Chengcheng Wang, Yujie Yang, Tianyu Guo, and Yunhe Wang. DenseMamba: State space models with dense hidden connection for efficient large language models. In *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics*, 2025.
- [32] Haruka Ezoe and Kazuhiro Sato. Model compression method for S4 with diagonal state space layers using balanced truncation, 2024.
- [33] Jindong Jiang, Xiuyu Li, Zhijian Liu, Muyang Li, Guo Chen, Zhiqi Li, De-An Huang, Guilin Liu, Zhiding Yu, Kurt Keutzer, et al. STORM: Token-efficient long video understanding for multimodal LLMs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, 2025.
- [34] Geewook Kim and Minjoon Seo. State-space hierarchical compression with gated attention and learnable sampling for hour-long video understanding in large multimodal models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

- [35] Jiaming Liu, Mengzhen Liu, Zhenyu Wang, Pengju An, Xiaoqi Li, Kaichen Zhou, Senqiao Yang, Renrui Zhang, Yandong Guo, and Shanghang Zhang. RoboMamba: Efficient vision-language-action model for robotic reasoning and manipulation. In *Advances in Neural Information Processing Systems*, 2024.
- [36] Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems*, 2017.
- [37] Maximilian Nickel and Douwe Kiela. Learning continuous hierarchies in the lorentz model of hyperbolic geometry. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [38] Hermann Karcher. Riemannian center of mass and mollifier smoothing. *Communications on Pure and Applied Mathematics*, 30(5):509–541, 1977.
- [39] Aaron Lou, Isay Katsman, Qingxuan Jiang, Serge Belongie, Ser-Nam Lim, and Christopher De Sa. Differentiating through the fréchet mean. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 6393–6403. PMLR, 2020.
- [40] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Ji Li, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: A progressive robustness benchmark for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.
- [41] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning, 2023.
- [42] Physical Intelligence. openpi: Open-source models and packages for robot learning. <https://github.com/Physical-Intelligence/openpi>, 2026. Software repository, accessed 2026-08-05.
- [43] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid Loss for Language Image Pre-Training, 2023.

A Mathematical Derivations

This appendix states the masked recurrence and residual-reader operators used by RS. Let $\tilde{s}_\tau^v$ and $\tilde{m}_\tau^v$ be the state and output proposed by the two-layer temporal Mamba module for view v . With availability indicator $c_\tau^v \in \{0, 1\}$, the implemented update is

$$s_\tau^v = c_\tau^v \tilde{s}_\tau^v + (1 - c_\tau^v) s_{\tau-1}^v, \quad m_\tau^v = c_\tau^v \tilde{m}_\tau^v + (1 - c_\tau^v) m_{\tau-1}^v. \quad (14)$$

Thus, an unavailable camera neither changes its recurrent cache nor overwrites its most recent memory token. For one attention head with valid-view mask c_t^v , the history reader computes

$$a_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d_h}) c_t^{v_j}}{\sum_{r=1}^V \exp(q_i^\top k_r / \sqrt{d_h}) c_t^{v_r}}, \quad \tilde{h}_i = h_i + \gamma W_O \sum_{j=1}^V a_{ij} v_j, \quad (15)$$

where all attention weights are defined as zero when no view is valid. The multi-head implementation applies Eq. 15 to each head and concatenates the resulting contexts. Finally, the four retrieved experience tokens are points on the Poincare geodesic

$$\Gamma_{q,p}(\tau) = \text{Exp}_q(\tau \text{Log}_q(p)), \quad \tau \in \{0.45, 0.55, 0.65, 0.75\}, \quad (16)$$

from the current query q to the prototype p of the retrieved lowest common ancestor. These points are decoded independently before entering the original VLM self-attention stack.

B Detailed Framework Description

B.1 Implementation Details of Mamba-Compressed Visual History

Backbone feature paths and adapters. The history pathway reuses the frozen π_0 visual encoder [1] through the shared openpi policy interface [42]. The frozen SigLIP [43] forward pass returns 16×16 patches through `pre_logits_2d`. After stop-gradient, an independently trained $\text{LayerNorm}(D_{\text{SigLIP}}) + \text{Linear}(D_{\text{SigLIP}}, 256)$ adapter maps these patches to the compressor width. Original VLM image tokens are unchanged. No PCA or auxiliary distillation objective is used. Training caches contain complete episode patch sequences and view masks, but never Mamba memories or recurrent states.

Compressor configuration and view masking. The compressor width is $d = 256$, with state size 16, convolution width 4, expansion factor 2, and head width 64. One Mamba layer operates in each spatial direction; aligned outputs are averaged, pooled by one learned query, and passed through a learned projection. Two causal temporal Mamba layers then update per-view histories. Temporal parameters are shared across the ordered multi-view stream, whereas convolution and SSM states remain separate for each view. For availability mask $c_\tau^v \in \{0, 1\}$, the temporal update is

$$(m_\tau^v, s_\tau^v) = \begin{cases} \text{Mamba}_{\text{temp}}^{(2)}(p_\tau^v, s_{\tau-1}^v), & c_\tau^v = 1, \\ (m_{\tau-1}^v, s_{\tau-1}^v), & c_\tau^v = 0, \end{cases} \quad (17)$$

so an unavailable view changes neither its recurrent state nor its latest memory representation.

Residual reader and backbone seams. The history reader uses four attention heads of width 64 and residual scale $\gamma = 0.05$. Memory masks remove unavailable views from the attention logits, and an item with no valid memory receives an exactly zero residual update. Disabling visual history bypasses the reader and preserves the original action path. The abstract H_t in Eq. 11 denotes the final hidden representations used for action prediction. For π_0 , it is instantiated by `suffix_out` before `action_out_proj` at every flow-matching step. This provides a single residual history-reading interface while retaining the π_0 policy parameters and independently trained RS modules.

Online state contract. At inference, each sampled frame performs one spatial encoding and one mask-conditional temporal update for every available view. Episode reset clears convolution caches, SSM states, latest view memories, and the sampling timer, so no state crosses episode boundaries. Recurrent-state shape is independent of episode length. Appendix D gives TBPTT and optimization details.

B.2 Formalization of the Hyperbolic Experience Space

Poincare geometry and numerical convention. We use curvature magnitude $c = 1$ and the open ball $\mathbb{B}^d$. For floating point tensors, $\epsilon = 10^{-7}$ for float32/bfloat16/float16 and 10^{-12} for float64. The implementation clamps `artanh` inputs to $[-1 + \epsilon, 1 - \epsilon]$ and projects points to radius $1 - 10\epsilon$. With $\lambda_x = 2/(1 - \|x\|^2)$ and Mobius addition $\oplus$, the common operations are

$$d_{\mathbb{B}}(x, y) = 2 \operatorname{artanh}(\| -x \oplus y \|), \quad (18)$$

$$\operatorname{Exp}_x(v) = x \oplus \left(\tanh \frac{\lambda_x \|v\|}{2} \frac{v}{\|v\|} \right), \quad \operatorname{Log}_x(y) = \frac{2 \operatorname{artanh}(\| -x \oplus y \|)}{\lambda_x \| -x \oplus y \|} (-x \oplus y). \quad (19)$$

Norms and denominators are clamped by ϵ , with continuous zero limits. A Karcher solve starts from the projected weighted Euclidean mean and runs 12 updates $m \leftarrow \operatorname{Exp}_m(0.5 \sum_i w_i \operatorname{Log}_m z_i)$.

P-VAE leaves and query. For every eligible final-layer hidden state $h_i \in \mathbb{R}^{D_{\text{VLM}}}$, the wrapped-normal P-VAE uses

$$z_i = \mu_i = \operatorname{Exp}_0(0.25 \tanh f_\mu(h_i)), \quad \sigma_i = \operatorname{softplus}(\operatorname{clamp}(f_\sigma(h_i), -8, 4)) + 10^{-4}, \quad (20)$$

and optimizes reconstruction plus β KL. Its decoder is an MLP on $\operatorname{Log}_0(\Pi(z))$. Sampling is used in training, whereas retrieval always uses projected posterior locations. The successful-episode filter is applied before the immutable snapshot is built. The online [MEM_QUERY] hidden follows the same $z = \mu_\phi(h)$ path to obtain q_t ; it has no additional head or cone semantics.

Compact HypHC-inspired triples and tree decode. Let $Z = (z_1, \dots, z_N)$ be P-VAE leaves and let $\rho = 0.75$. An independent auxiliary copy is initialized as

$$y_i = \Pi \left(\rho \frac{z_i}{\max(\|z_i\|, 10^{-6})} \right), \quad (21)$$

where Π is the Poincare-ball projection. Gradients through this phase do not modify Z . We form $s_{ij} = \exp[-d_{\mathbb{B}}^2(z_i, z_j)/(2b^2)]$, where b is the median nonzero pairwise P-VAE distance, and sample

distinct triples (a, b, c) . For the three pairs $\mathcal{P} = \{(a, b), (a, c), (b, c)\}$, the implemented compact loss is

$$w_{ij} = \frac{\exp[-d_{\mathbb{B}}(y_i, y_j)]}{\sum_{(r,s) \in \mathcal{P}} \exp[-d_{\mathbb{B}}(y_r, y_s)]}, \quad \mathcal{L}_{\text{tri}} = \mathbb{E} \left[\left(\sum_{(i,j) \in \mathcal{P}} s_{ij} - \sum_{(i,j) \in \mathcal{P}} w_{ij} s_{ij} \right)^2 \right]. \quad (22)$$

After each update, y_i is reprojected to radius ρ , subject to the numerical floor in Eq. 21. This is a differentiable triplet relaxation inspired by HypHC, rather than the full HypHC training system.

All leaf pairs are sorted by descending dot product, with stable leaf-ID ties. A Union-Find single-link pass merges disconnected components until the deterministic binary tree has $2N - 1$ nodes. The decoded root serves as a virtual router. For retrieval-space initialization, each internal prototype is recomputed as the Karcher mean of its descendant P-VAE leaves z_i ; equal-radius y_i values remain auxiliary.

Topology-only cone calibration. For $K = 0.1$, a non-root internal prototype p has half-aperture and exclusion radius

$$\psi(p) = \arcsin \left(K \frac{1 - \|p\|^2}{\|p\|} \right), \quad r_{\min} = \frac{2K}{1 + \sqrt{1 + 4K^2}}. \quad (23)$$

The angle is computed between $-\text{Log}_p(0)$ and $\text{Log}_p(x)$, with cosine clamped to $[-1 + 10^{-7}, 1 - 10^{-7}]$, and $E_{\text{cone}}(p, x) = [\angle_p(x) - \psi(p)]_+$. If $\|p\| < r_{\min}$, this energy is marked invalid and routing uses distance only.

Topology supplies positives from non-root internal-to-child edges and internal-to-descendant-leaf closure. For each positive parent, outside-subtree leaves with the lowest initial cone energy supply hard negatives. With an adaptive radial margin m_{px} , calibration minimizes

$$\begin{aligned} \mathcal{L}_{\text{cal}} = & \lambda_+ \mathbb{E}_{(p,x) \in \mathcal{P}} E_{\text{cone}}(p, x) + \lambda_- \mathbb{E}_{(p,n) \in \mathcal{N}} [\gamma - E_{\text{cone}}(p, n)]_+ \\ & + \lambda_r \mathbb{E}_{(p,x) \in \mathcal{P}} [\|p\| + m_{px} - \|x\|]_+ + \lambda_a \mathbb{E}_p d_{\mathbb{B}}^2(p, p^{(0)}). \end{aligned} \quad (24)$$

Relations are generated solely from the decoded topology and never read task, subtask, or action labels. Optimization variables are tangent coordinates for non-root internal prototypes only. P-VAE leaves and the virtual root are copied back exactly after optimization.

Beam and exact retrieval. For a non-root internal node v , the routing score is

$$s_{\text{node}}(v \mid q_t) = 0.25 d_{\mathbb{B}}(p_v, q_t) + E_{\text{cone}}(p_v, q_t). \quad (25)$$

The approximate search begins at the virtual root’s children, retains the eight lowest-scoring internal nodes at each level, and expands their children. Visited leaves accumulate until the usable pool reaches $\max(8, \lceil 2 \cdot 8 \rceil) = 16$. A visited leaf ℓ is ranked by

$$s_{\text{leaf}}(\ell \mid q_t) = d_{\mathbb{B}}(q_t, z_\ell) + \frac{1}{|\mathcal{A}_\ell^{\text{valid}}|} \sum_{v \in \mathcal{A}_\ell^{\text{valid}}} E_{\text{cone}}(p_v, q_t), \quad (26)$$

where the second term is zero if no ancestor has a valid cone. After excluding records whose episode ID equals the query episode, the best eight leaves are returned. If exclusions leave fewer than the required number, an explicit exact safety completion ranks all leaves.

The exact oracle batches all non-root internal cone energies, propagates ancestor sums and valid counts with one root-to-leaf DFS, batches all leaf distances, and performs the same final top-8 sort. Its per-query cost is $\mathcal{O}(N)$. The beam records visited and pruned nodes and its fallback reason, but has no sublinear worst-case guarantee and may also cost $\mathcal{O}(N)$.

Composer and fixed output. The composer finds the lowest common internal ancestor a_t of the returned top-8 leaves in the decoded topology and obtains its calibrated P-VAE-space prototype p_{a_t} . For zero-based slot j , the implementation uses

$$(\tau_0, \tau_1, \tau_2, \tau_3) = (0.45, 0.55, 0.65, 0.75), \quad u_j = \text{Exp}_{q_t}(\tau_j \text{Log}_{q_t} p_{a_t}). \quad (27)$$

Decoding $u_0, \dots, u_3$ yields exactly $4 \times D_{\text{VLM}}$ values. Empty candidates return four zero decoder-width slots to the non-blocking caller but are classified as empty retrieval and never published by the asynchronous worker.

Asynchronous validity. One global capacity-one request queue uses latest-wins submission. Every submission replaces any queued request, increments a sequence number, and records the latest tuple (g, s, e, t) of generation, sequence, episode, and step. A completed worker result is published only when its tuple still matches the latest request and elapsed wall time is within the configured timeout. Empty results, exceptions, and post-completion timeouts increment telemetry and are discarded. The action loop never waits. Poll validity is

$$\text{valid}(\mathcal{R}_\tau, t) = \mathbb{I}[e_\tau = e_t] \mathbb{I}[g_\tau = g_t] \mathbb{I}[1 \leq t - \tau \leq 2]. \quad (28)$$

Thus, an age-zero result cannot affect the current action step. Reset increments the generation, clears episode results and step state, and drains the queue. Search also excludes the query episode before ranking.

Prompt attention contract. For each batch row, four projected $\langle \text{exp} \rangle$ embeddings are inserted immediately before the first real image embedding, and one learned [MEM_QUERY] embedding is inserted at the end of the task prompt. Position IDs are recomputed from the valid-token mask, and decoder-only adapters use the corresponding causal or prefix-block attention edges. The existing VLM processes this augmented prompt with its original self-attention. From the final hidden sequence, [MEM_QUERY] is detached and submitted for use only at $t + 1$ or later; original token positions are gathered back before the action expert.

No production experience cross-attention or second residual gate exists. With `enabled=false`, no virtual tokens are inserted and inputs, attention mask, and original action path are returned unchanged. With experience enabled but no memory yet, masked empty experience slots and a valid [MEM_QUERY] are still inserted so that a future query can be produced.

C Experimental Configuration and Additional Results

C.1 Complete Evaluation Protocol

For every paired comparison, evaluation instances and initial states are fixed before either policy is run. LIBERO-Plus uses all 10,030 single-perturbation instances described in Section 5. The reported primary setting trains only on original LIBERO; the released LIBERO-Plus training set is not used. The standard LIBERO component study uses each suite’s demonstrations for the corresponding Spatial, Object, Goal, or Long evaluation.

The frozen-backbone study trains the SigLIP dimension adapter, bidirectional spatial Mamba blocks, learned-query pooling and projection, two temporal Mamba layers, the residual history reader, and the implemented experience components: P-VAE, independent topology-learning coordinates, internal-prototype calibration, prompt projection, and [MEM_QUERY] embedding. It includes no query-radius head, experience cross-attention, or second residual gate.

C.2 Mamba Training-Window and Fixed-State Streaming Evaluation

For the history-length study, we train VMC with Mamba histories of $H \in \{8, 16, 32, 64, 128\}$ and evaluate the LIBERO-Long success-rate change relative to a no-Mamba baseline every 5k steps. The TBPTT study fixes $H = 64$ and varies the backpropagation truncation length over $\{4, 8, 16, 24, 32\}$. Training loss is logged throughout optimization. The main paper reports the resulting success-rate and loss curves; the supplement contains additional optimization diagnostics.

C.3 Component Ablation Details

The component ablation uses only π_0 and compares the base policy, π_0 +VMC, and full π_0 +RS. Each variant is trained and evaluated under the standard protocol for LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long. No variant changes the demonstrations, data order, optimization budget, checkpoint-selection rule, action parameterization, or evaluation instances within a suite. The grouped plot reports each suite and their unweighted Overall mean. The base π_0 result is a single run; VMC and RS report five runs, with standard deviation shown in the main-paper figure.

C.4 P-VAE and HypHC Visualization Data

Figure 3 uses the first 300 successful π_0 inference records from <https://huggingface.co/datasets/physical-intelligence/libero>. The records span the following ten tasks:

Task	Instruction
T00	Put the white mug on the left plate and put the yellow and white mug on the right plate.
T01	Put the white mug on the plate and put the chocolate pudding to the right of the plate.
T02	Put the yellow and white mug in the microwave and close it.
T03	Turn on the stove and put the moka pot on it.
T04	Put both the alphabet soup and the cream cheese box in the basket.
T05	Put both the alphabet soup and the tomato sauce in the basket.
T06	Put both moka pots on the stove.
T07	Put both the cream cheese box and the butter in the basket.
T08	Put the black bowl in the bottom drawer of the cabinet and close it.
T09	Pick up the book and place it in the back compartment of the caddy.

C.5 Real-Robot Task Definitions and Success Criteria

The real-robot suite probes four memory-centric requirements: counting repeated events, preserving an ordered action sequence, grounding a persistent object name, and composing these abilities over a long horizon. Occluded placement is an additional perceptual stressor. The task instructions are as follows:

- **Press-2:** *Press the purple button twice.* Press-4 and Press-6 use the same button with the requested count changed to four and six.
- **Order (Press-hard):** *First, press the blue button once. Then press the purple button twice. Finally, press the pink button three times.*
- **Occl.:** *First, put the yellow ball into the green cup, then place the green cup into the red box.*
- **Chen.-S:** *Move Chenchen from the yellow box to the green box.* Here, Chenchen always denotes the designated plush toy rather than a distractor.
- **Chen.-H:** *First, move Chenchen from the yellow box to the green box. Then press the blue button once. Next, put the yellow ball into the green cup, and place the green cup into the yellow box. Finally, press the pink button once.*

Strict success requires completion of every requested subgoal in the specified order. Any wrong object, wrong order, extra button press, safety stop, timeout, or human intervention is a failure.

Training and generalization conditions. Training uses Press-2, Ordered Button Pressing, Occluded Placement, and the Chenchen-simple condition. Press-4, Press-6, and Chenchen-hard are evaluated without additional task-specific training.

Each physical condition is evaluated over 10 or 25 trials. In addition to strict success rate, logging records button-count error, the first sequence error, redundant actions after hidden placement, completed-subgoal fraction, and the first failure stage.

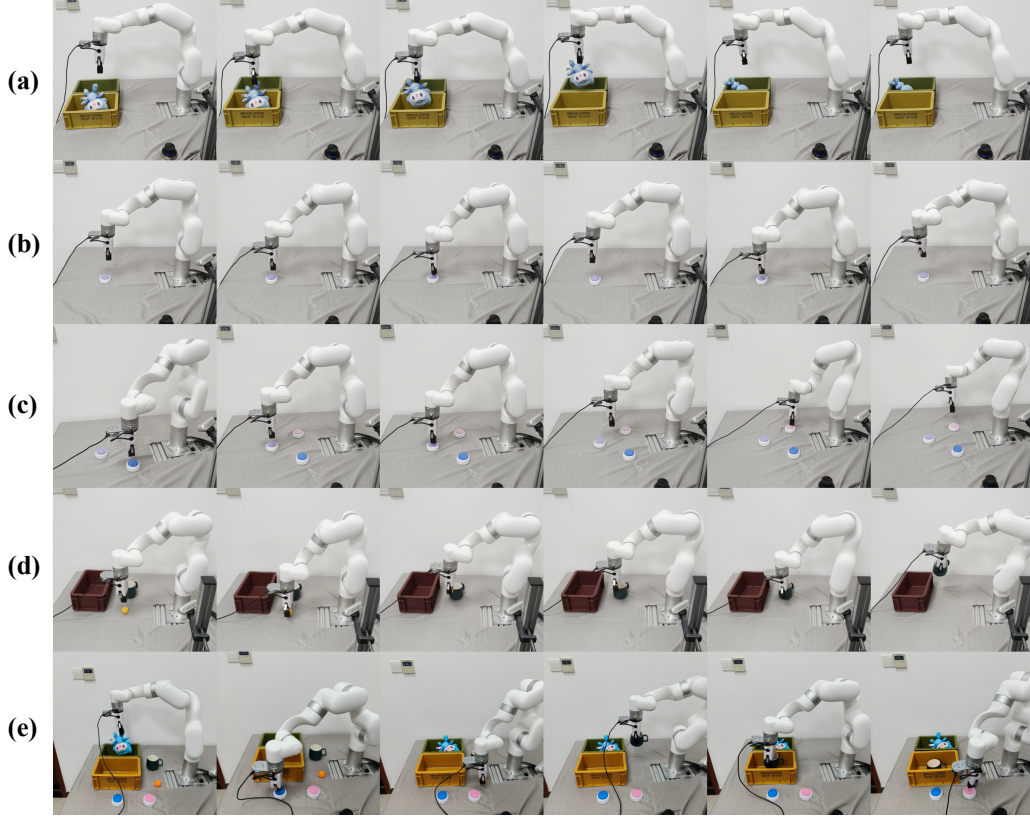


Figure 7: Representative successful keyframe sequences for the real-robot task families: (a) Chen.-S name grounding, (b) Press-2 counting, (c) Order sequence following, (d) occluded placement, and (e) Chen.-H compositional long-horizon execution.

D Training Configuration

RS is trained through the pinned openpi π_0 interface. The released π_0 policy checkpoint initializes the policy, while the visual compressor starts from the seed-0 mamba_base; no additional standalone Mamba pretraining is used. Cached inputs contain frozen SigLIP pre_logits_2d patch features and view masks, rather than cached recurrent activations. Each example therefore reconstructs its causal history using the current RS parameters.

Table 3: Default π_0 +RS training configuration.

Item	Setting
Visual history	3 views, 1 FPS, 64 frames; TBPTT window 16
VMC	$d = 256$, state 16, convolution 4, expansion 2, head width 64
Precision / parallelism	bf16 activations; fp32 parameters and gradients; 4-device FSDP
Optimizer	Grouped AdamW; weight decay 10^{-4} ; gradient clip 1; EMA 0.999
Batch and checkpoints	Global batch 32; checkpoint every 2,500 steps
Stage A	5k steps; RS adapter, VMC, and history reader trainable
Stage B	25k steps; RS modules and action-side parameters trainable; PaliGemma prefix frozen
Learning rates	Stage A: 2×10^{-5} (Mamba), 10^{-4} (adapter/reader); Stage B: 10^{-5} (Mamba/adapter), 5×10^{-5} (reader/action)

Each nonzero learning-rate group uses a 100-step warm-up followed by cosine decay to one tenth of its peak rate. The default evaluation uses a 64-frame history. The history-length and TBPTT studies reported in Figure 6 vary those values separately; they are not part of the default training setting.